

D O C U M E N T A Ç Ã O T É C N I C A

EKS para Terraform

Solução find-or-create para recriação e DR de clusters, na
mesma conta ou cross-account

Documentação técnica completa da solução que captura um cluster **Amazon EKS** existente e gera um projeto **Terraform modular e portátil** capaz de recriar esse cluster na **mesma conta** (recuperação/DR) ou em **outra conta** (migração/DR cross-account), reutilizando a infraestrutura de base que já existe no destino e criando apenas o que falta.

Gerado em 18/06/2026

Sumário

1. Visão geral
2. Os dois produtos: CLI e Lambda
3. As três fases: captura, geração, aplicação
4. O mecanismo find-or-create
 1. Find-or-create vs. find-or-skip
5. O helper `aws_lookup.py` (data.external)
6. A tag-marca e a idempotência
7. As flags de controle
 1. `disable_resource_reuse` (default false)
 2. `recreate_external_routes` (default true, só aparece quando há alvos externos)
8. Tradução de IDs e ARNs entre contas
9. O `vpc_found` como interruptor-mestre da rede
10. Transit Gateway cross-account
11. O que é capturado e recriado (cobertura)
12. Cenários de uso
 1. Cenário A: mesma conta, só o cluster foi deletado
 2. Cenário B: mesma conta, tudo foi deletado
 3. Cenário C: outra conta, landing zone com base existente
 4. Cenário D: outra conta, conta limpa
 5. Tabela-resumo dos cenários
13. Passo a passo operacional do apply
14. Estrutura dos arquivos gerados
15. Permissões IAM
16. Idempotência e reexecução
17. Limitações conhecidas
18. Troubleshooting / FAQ

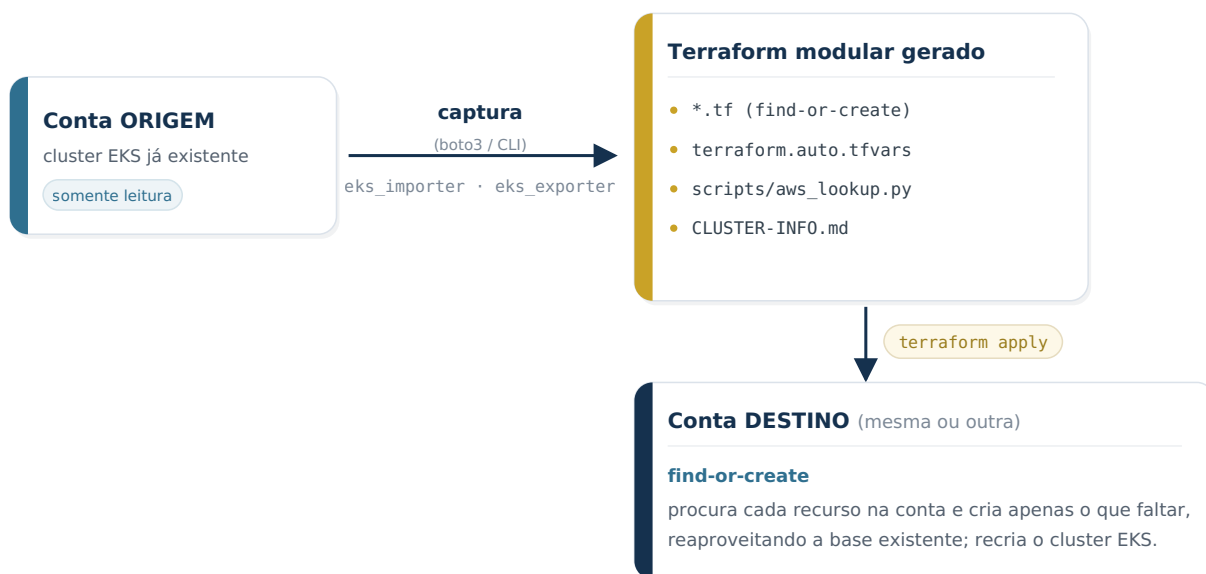
1. Visão geral

O problema que a solução resolve: você tem um cluster EKS criado fora do Terraform (no console, por scripts, etc.) e precisa de uma forma versionada e reproduzível de recriá-lo, seja para recuperação de desastre na mesma conta, seja para clonar/migrar para outra conta.

A ideia central é a estratégia **find-or-create**: o Terraform gerado não assume que o destino está vazio nem que está cheio. Para cada recurso de base (VPC, subnets, security groups, IAM roles, key pairs, Transit Gateway, etc.), em tempo de `plan` ele procura o equivalente na conta de destino e:

- se já existir → reutiliza o ID/ARN existente (não recria);
- se não existir → cria um idêntico ao da origem.

Isso permite que o mesmo conjunto de arquivos funcione em qualquer um dos cenários, com pouca ou nenhuma edição.



2. Os dois produtos: CLI e Lambda

A mesma lógica de captura/geração existe em dois empacotamentos:

	eks_importer (CLI)	eks_exporter (Lambda)
Execução	<code>python -m eks_importer <cluster> [região] [profile]</code>	AWS Lambda (<code>lambda_handler</code>)
Acesso à AWS	AWS CLI (subprocess)	boto3 (mesmas estruturas JSON)
Saída	grava pasta <code>terraform-<cluster>/</code> em disco	gera tudo em memória e faz push no GitLab
Onde roda	máquina do operador	agendada/event-driven na conta de origem

Os módulos de coleta, templates HCL, helpers e o `aws_lookup.py` são idênticos entre os dois; só mudam a borda de I/O (disco vs. memória+GitLab) e o runner (CLI vs. boto3). No Lambda, o `run_aws_command` traduz a string `aws ec2 describe-...` para a chamada boto3 equivalente, então toda a lógica a jusante é a mesma.

O artefato gerado (os `.tf`, o `tfvars`, o `aws_lookup.py`) é idêntico nos dois caminhos. Esta documentação descreve esse artefato e seu comportamento no `terraform apply`, que é onde toda a inteligência cross-account acontece.

3. As três fases: captura, geração, aplicação

Fase 1, Captura (na conta de ORIGEM). A ferramenta lê (samente leitura) a configuração completa do cluster e da infraestrutura que ele usa: cluster, node groups, Fargate, add-ons, pod identities, access entries,

Auto Mode, e toda a base (VPC, subnets, route tables, rotas, NAT, IGW, DHCP, endpoints, Transit Gateway, IAM roles/policies/users, security groups, launch templates, key pairs). Tudo é serializado em `terraform.auto.tfvars.json`.

Fase 2, Geração. A ferramenta escreve os arquivos `.tf` modulares (templates estáticos), o `terraform.auto.tfvars.json` (dados da origem), o helper `scripts/aws_lookup.py` e o relatório `CLUSTER-INFO.md` (com avisos e limitações específicos do seu cluster).

Fase 3, Aplicação (na conta de DESTINO). Você roda `terraform init/plan/apply`. Durante o `plan`, os `data.external` chamam o `aws_lookup.py`, que consulta a conta de destino e decide, recurso por recurso, o que reutilizar e o que criar. Durante o `apply`, o que falta é criado e o cluster EKS é recriado.

Importante: a captura roda com credenciais da origem; o apply roda com credenciais do destino. Por isso o `aws_profile` no tfvars vem vazio — ele é o profile do destino, que você preenche (ou deixa vazio para a cadeia padrão de credenciais). Nunca coloque o profile da origem ali, ou os lookups consultariam a conta errada.

4. O mecanismo find-or-create

A ideia em uma frase: para cada recurso de base, antes de criar qualquer coisa, o Terraform pergunta à conta de destino "isto já existe aqui?". Se existe, reaproveita; se não, cria um igual ao da origem. Quem responde a essa pergunta é o `aws_lookup.py`, chamado em tempo de `plan`.

Vamos seguir um recurso de verdade do começo ao fim. Os exemplos desta seção saem do tfvars do cluster `cluster-xxxx` (`source_account_id 5000000000`); o conjunto exato de recursos muda de cluster para cluster, então trate os nomes abaixo como o que este cluster capturou, não como uma lista fixa. O recurso escolhido é a role `eksServiceRole` (a role do cluster, que no tfvars aparece como `"cluster_role_arn": "arn:aws:iam::5000000000:role/eksServiceRole"`). Suponha que a conta de destino seja `111122223333`.

Cada cluster captura o seu próprio conjunto. As listas de recursos nesta seção são específicas do `cluster-xxxx`. Outro cluster terá outros nomes e pode nem ter alguns destes recursos: por exemplo, `AmazonEKSFargatePodExecutionRole` só existe aqui porque este cluster usa Fargate (tem `fargate_profiles`); um cluster sem Fargate não terá essa role. Da mesma forma, `pod-role-xxxx`, `app-user`, `lt-workers-xxxx`, etc. são deste ambiente. O mecanismo find-or-create é idêntico para qualquer cluster; só muda o conteúdo do `terraform.auto.tfvars.json`.

Passo 1, a pergunta (roda no `plan`, na conta de destino). Para cada role que este cluster capturou (aqui: `eksServiceRole`, `eksWorkerNodeRole`, `AmazonEKSFargatePodExecutionRole` por causa do Fargate, mais as roles de Pod Identity como `pod-role-xxxx`, `ClusterAutoscalerRole`, etc.), o Terraform chama o lookup perguntando "existe uma role com este nome aqui?". A chave de busca da role é o nome (não o ARN, porque o ARN muda de conta).

Passo 2, a decisão. O resultado vira um booleano `iam_role_found["eksServiceRole"]`: é `true` se o lookup achou a role no destino (e `disable_resource_reuse` é `false`), e `false` caso contrário.

Passo 3, criar só se faltar. O recurso `aws_iam_role.this["eksServiceRole"]` só é criado quando `found` é `false`. Os dois desfechos:

- O destino já tem `eksServiceRole` (ex.: landing zone padronizada): `found = true`, a role não é recriada, e o cluster passa a usar a role que já existe lá.
- O destino não tem: `found = false`, a role é criada idêntica à da origem (mesmas managed policies, mesma trust, mesmas inline policies) e recebe a tag-marca `eks-importer:stack = cluster-xxxx`.

Passo 4, a tradução (o que o cluster realmente consome). O ARN da role muda entre contas, então existe uma tabela `iam_role_arn_by_name` que reescreve o número da conta:

	Na origem (tfvars)	No destino (efetivo)
Nome	<code>eksServiceRole</code>	<code>eksServiceRole</code> (igual)
ARN	<code>arn:aws:iam::50000000000:role/eksServiceRole</code>	<code>arn:aws:iam::111122223333:role/eksServiceRole</code>
Se já existia no destino	n/a	reutiliza essa role
Se não existia	n/a	cria com a tag <code>eks-importer:stack=cluster-xxxx</code>

O `aws_eks_cluster` então usa `iam_role_arn_by_name["eksServiceRole"]` em vez do ARN literal da origem. Detalhe importante: esse ARN é calculado por substituição de texto (troca `50000000000` pela conta de destino), não lendo a role criada. Isso é de propósito: se o ARN dependesse de "achou ou criou", o valor viraria "known after apply" e o Terraform substituiria o cluster à toa. Como o nome é fixo, o ARN no destino é previsível.

Em HCL, resumido, é este quarteto (com os valores reais do seu cluster):

```
# 1. PERGUNTA -> roda no plan, consulta a conta de DESTINO
data "external" "iam_role_lookup" {
  for_each = var.iam_roles      # "eksServiceRole", "eksWorkerNodeRole", ...
  query = {
    kind      = "iam_role"
    name      = each.key        # ex.: "eksServiceRole"
    exclude_value = "cluster-xxxx" # ignora o que ESTA stack criou (tag-marca)
  }
}

# 2. DECISAO -> achou no destino e o reuso esta ligado?
locals {
  iam_role_found = { for n, r in var.iam_roles : n =>
    var.disable_resource_reuse ? false : (data.external.iam_role_lookup[n].result.arn != "") }
}

# 3. CRIA so se NAO achou -> eksServiceRole so nasce se o destino nao tiver
resource "aws_iam_role" "this" {
  for_each = { for n, r in var.iam_roles : n => r if !local.iam_role_found[n] }
  name     = each.key
  tags     = { "eks-importer:stack" = "cluster-xxxx" } # a marca
}

# 4. TRADUCAO -> ARN com a conta de destino; e o que o cluster consome
locals {
  iam_role_arn_by_name = { for n, r in var.iam_roles : n =>
    replace(r.source_arn, var.source_account_id, data.aws_caller_identity.current.account_id) }
}
```

O mesmo mecanismo, mas para a VPC (o interruptor da rede). A VPC do seu cluster é `vpc-xxxx` , CIDR `x.x.x.x/xx` . A chave de busca dela é o CIDR. O lookup pergunta "existe uma VPC `x.x.x.x/xx` no destino?":

	Origem	Destino (achou)	Destino (não achou)
VPC	<code>vpc-xxxx (x.x.x.x/xx)</code>	a VPC existente	uma VPC nova
<code>vpc_found</code>	n/a	true	false

	Origem	Destino (achou)	Destino (não achou)
Subnets, route tables, NAT, IGW, TGW, endpoints	n/a	intocados (já existem)	reconstruídos do backup

Ou seja, a mesma pergunta-resposta da role vale para a VPC, mas com uma consequência muito maior: se a VPC é reaproveitada, toda a malha de rede dela é assumida pronta e nada de rede é recriado (ver §9). Por isso a VPC é o "interruptor-mestre".

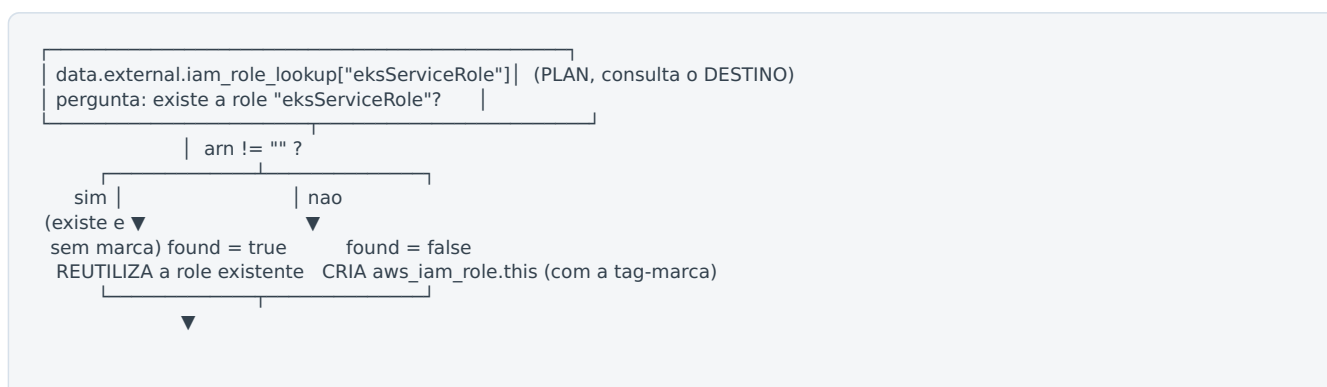
Resumindo as quatro peças (com os nomes que você vê nos `.tf`):

1. `data.external.<x>_lookup` : a pergunta ao destino (ex.: `iam_role_lookup` , `vpc_lookup` , `sg_lookup`). Devolve o ID/ARN achado ou `""` .
2. `local.<x>_found` : a decisão (lookup achou e `disable_resource_reuse=false`).
3. `resource.<x>` com `for_each / count` gateado por `!found` : cria só o que falta.
4. `local.<x>_id_by_source` (ou `..._arn_by_name`) : a tabela de tradução que o cluster, nodegroups, fargate, SGs e rotas consomem no lugar do ID/ARN literal.

As chaves de busca por tipo (a coluna de exemplos é do `cluster-xxxx`):

Recurso	Chave de busca	Exemplo no cluster-xxxx
VPC	CIDR primário	<code>x.x.x.x/xx</code>
Subnet	(vpc-id + cidr-block)	<code>subnet-xxxx = x.x.x.x/xx</code>
Security Group	(vpc-id + group-name)	<code>sg-xxxx = nome sg-app-1</code>
Internet Gateway	vpc-id	<code>igw-xxxx</code>
IAM Role	nome	<code>eksServiceRole</code> , <code>eksWorkerNodeRole</code>
IAM Policy (customer-managed)	nome	<code>ClusterAutoscalerPolicy</code>
IAM User	nome	<code>app-user</code> , <code>brian.richard</code>
Key Pair EC2	nome	<code>keypair-xxxx</code>
Launch Template	nome	<code>lt-workers-xxxx</code>
Transit Gateway	ID via RAM → Name tag → cria novo (ver §10)	<code>tgw-xxxx (Name xxxx-TGW)</code>

Esquema da decisão, com o exemplo da role:



```
iam_role_arn_by_name["eksServiceRole"] = arn:aws:iam::<conta-destino>:role/eksServiceRole
consumido por aws_eks_cluster.role_arn
```

Find-or-create vs. find-or-skip

Os recursos de base acima usam **find-or-create**: se não existirem no destino, são criados. Já os alvos de rota externos (peering, VPN gateway, carrier gateway) usam uma variante, o **find-or-skip**: a stack não cria esses recursos (têm segredo, dependem de aceitação cross-account ou pertencem a outro appliance), então o lookup só procura o equivalente no destino. Se achar, a rota aponta para ele; se não achar, a rota é simplesmente pulada (omitida), sem derrubar o `apply`. A ENI é sempre pulada (a interface pertence a um appliance/LB que a stack não gerencia). Isso é o que permite manter `recreate_external_routes = true` mesmo cross-account sem risco de falha.

Alvo de rota externo	Chave de busca (find-or-skip)	Se não achar
Peering	peering ativa, uma ponta = nossa VPC, outra ponta cobre o CIDR da rota	rota pulada
VPN Gateway (VGW)	VGW anexado à nossa VPC (<code>attachment.vpc-id</code>)	rota pulada
Carrier gateway	carrier gateway da nossa VPC (<code>vpc-id</code>)	rota pulada
ENI	(sem busca, sempre pulada)	rota pulada
Core network (Cloud WAN)	ARN literal (vale se compartilhado via RAM)	apply falha se ausente

5. O helper `aws_lookup.py` (`data.external`)

É o motor do find-or-create. Roda em tempo de `terraform apply` (no `plan`), na máquina/runner que executa o Terraform (não na Lambda de captura), e por isso usa o AWS CLI autenticado na conta de destino.

Por que `data.external` e não os data sources nativos (`aws_vpcs`, `aws_iam_role`)? Porque a documentação do `aws_vpcs` diz textualmente "this data source will fail if none are found", ou seja, o data source nativo quebra o `apply` quando não encontra nada, que é justamente o caso "conta nova, ainda não tem a VPC". O AWS CLI, ao contrário, devolve lista vazia sem erro. O external data source apoiado neste script sempre termina com sucesso, devolvendo o ID encontrado ou vazio.

Protocolo (hashicorp/external):

- Lê um objeto JSON do stdin (os argumentos de `query`, todos strings).
- Escreve no stdout um objeto JSON cujos valores também são todos strings.
- Sai com status 0 em sucesso; status $\neq$ 0 + mensagem no stderr em erro real.

Entrada/saída, exemplos:

```
stdin : {"kind":"vpc","region":"xx-xxxx-1","cidr":"x.x.x.xx","exclude_value":"cluster-xxxx"}
stdout: {"id":"vpc-xxxx"} ou {"id":""}

stdin : {"kind":"iam_role","name":"eksServiceRole","exclude_value":"cluster-xxxx"}
stdout: {"arn":"arn:aws:iam::111122223333:role/eksServiceRole"} ou {"arn":""}
```

Cada kind tem um handler (`lookup_vpc`, `lookup_subnet`, `lookup_security_group`, `lookup_internet_gateway`, `lookup_iam_role`, `lookup_iam_policy`, `lookup_iam_user`, `lookup_instance_profile`, `lookup_key_pair`, `lookup_launch_template`, `lookup_transit_gateway`, e os de find-or-skip de rota `lookup_vpn_gateway`, `lookup_peering`, `lookup_carrier_gateway`). "Não encontrado" no IAM aparece como `NoSuchEntity`; o helper

trata isso como vazio (→ criar), e propaga apenas erros reais (ex.: `AccessDenied`), que abortam o apply de propósito (melhor falhar que gerar infra incompleta silenciosamente).

6. A tag-marca e a idempotência

Todo recurso de base que a stack cria recebe a tag `eks-importer:stack = <nome-do-cluster>`. O `aws_lookup.py` ignora recursos que carregam essa marca igual ao cluster atual.

Isso é o que torna o find-or-create idempotente automaticamente. Sem a marca, haveria um bug grave na segunda execução:

```
apply #1 (destino limpo):
lookup VPC x.x.x.x/xx -> nao acha -> vpc_found=false
-> CRIA aws_vpc.this (vira vpc-0NOVO..., marca aplicada)
state agora tem aws_vpc.this

apply #2 (mesmo state):
lookup VPC x.x.x.x/xx -> ACHARIA a vpc-0NOVO... que a propria stack criou...
-> mas ela tem a marca eks-importer:stack=cluster-xxxx
-> o lookup a IGNORA -> retorna "" -> vpc_found=false
aws_vpc.this continua no config e no state -> SEM mudanca, SEM destruicao
```

Se o lookup não ignorasse a marca, no apply #2 ele acharia a própria VPC → `vpc_found=true` → `aws_vpc.this` com count 0 → o Terraform destruiria a VPC que acabou de criar. A marca previne exatamente isso.

Resumo: recurso pré-existente (sem a marca) → reutilizado; recurso criado pela stack (com a marca) → ignorado pelo lookup, mas gerenciado pelo state (nunca "reencontrado" e destruído); recurso inexistente → criado.

7. As flags de controle

Duas variáveis no `terraform.auto.tfvars.json` controlam o comportamento.

`disable_resource_reuse` (default false)

Liga/desliga o find-or-create globalmente.

- **false (padrão):** find-or-create ativo. `X_found` depende do lookup. Reutiliza o que existe; cria o que falta. Idempotente via tag-marca.
- **true:** desliga o reuso. `X_found = false` para todos os recursos de base → o Terraform cria e gerencia todos eles. Use quando o destino está limpo (recuperação na mesma conta após apagar a base, ou conta nova vazia) e você quer forçar que tudo seja gerenciado.

Com `true`, se algum desses recursos já existir no destino, o apply falha por conflito de nome (`EntityAlreadyExists`, `InvalidGroup.Duplicate`, etc.). Por isso só use em destino comprovadamente limpo. Em qualquer dúvida, deixe `false` — o find-or-create lida graciosamente com "alguns existem, outros não".

`recreate_external_routes` (default true, só aparece quando há alvos externos)

Controla rotas e attachments cujo alvo é externo/persistente: Transit Gateway, VPC peering, VPN gateway (VGW), ENI, carrier gateway, core network.

- **true (padrão):** recria essas rotas. Para TGW, dispara o find-or-create (ver §10). Para peering/VGW/carrier, dispara o find-or-skip (resolve no destino ou pula a rota). ENI é sempre pulada. Só core network usa ARN literal (vale se compartilhado via RAM).

- **false:** não cria nenhuma rota/attachment externo. O cluster sobe sem essa conectividade; você a configura depois.

Quando usar cada valor:

Situação	recreate_external_routes
Recuperação na mesma conta (os alvos externos ainda existem)	true
Outra conta, TGW compartilhado via RAM ou com mesmo Name tag	true
Outra conta, você quer o TGW replicado (a tool cria um novo)	true
Outra conta, com rotas de peering/VGW/carrier	true (find-or-skip resolve no destino; o que não existir é pulado)
Outra conta, a conectividade externa será montada à parte	false

Sem tensão de flag (resolvido por find-or-skip): em versões anteriores, `recreate_external_routes = true` era arriscado em outra conta porque peering/VGW/ENI usavam o ID literal da origem e o apply falhava. Isso foi resolvido: peering, VGW e carrier agora são find-or-skip (consultados na conta de destino; se o equivalente existir, a rota aponta para ele; se não, a rota é pulada, sem derrubar o apply). ENI é sempre pulada (a interface pertence a um appliance/LB que a stack não gerencia). Só o core network (Cloud WAN) ainda usa ARN literal, que funciona quando o core network está compartilhado via RAM (o caso comum). Resultado: pode manter `recreate_external_routes = true` mesmo cross-account; o TGW é resolvido/criado e as demais rotas externas são resolvidas-ou-puladas, nunca falham o apply (exceto core_network não compartilhado). Detalhes de cada rota afetada saem no `CLUSTER-INFO.md`.

8. Tradução de IDs e ARNs entre contas

IDs de rede (VPC, subnet, SG) e ARNs de IAM mudam entre contas. A solução resolve isso com tabelas de tradução em `locals.tf` / `iam.tf` e um token textual.

IDs de rede: `subnet_id_by_source`, `sg_id_by_source` mapeiam o ID de origem para o ID efetivo no destino (encontrado ou criado). Cluster, nodegroups e fargate consomem essas tabelas em vez do ID literal.

ARNs de IAM: o ARN embute o account-id, então muda entre contas. A tradução é determinística e feita por nome:

```
iam_role_arn_by_name[nome] = replace(source_arn, source_account_id, conta_destino)
```

Ou seja, pega o ARN de origem e troca o account-id da origem pelo da conta de destino (resolvido em apply via `data.aws_caller_identity.current`). Mesma conta → ARN idêntico. Isso é deliberado: derivar o ARN de `found ? data... : resource...` faria o valor virar "known after apply" quando o status found/created mudasse, forçando a substituição do cluster (porque `compute_config.node_role_arn` mudaria). Como o nome da role é fixo, o ARN no destino é previsível e estável.

Token `__DEST_ACCOUNT__`: documentos que embutem o account-id da origem (trust policies, documentos de policy customer-managed, policies de gateway/interface endpoints) têm o account-id da origem reescrito para o token `__DEST_ACCOUNT__` na captura; no apply, o Terraform troca o token pela conta de destino real (`data.aws_caller_identity.current.account_id`).

Saneamento de trust policy (portabilidade): a `assume_role_policy` de cada role é higienizada automaticamente para funcionar no destino:

- principais órfãos (IDs únicos `AIDA...` / `AR0A...` de usuários deletados) são removidos, a AWS rejeita esses;
- provedores Federated OIDC/SAML da origem são removidos, o provedor pertence ao cluster de origem e não existe no destino;
- ARNs de role/user da própria conta de origem são descartados (vestigiais quando a role usa Pod Identity) ou têm a conta reescrita;
- account-ids puros da origem viram `__DEST_ACCOUNT__`; ARNs de outras contas são mantidos (cross-account é válido);
- roles que são alvo de Pod Identity recebem a trust `pods.eks.amazonaws.com`.

No `cluster-xxxx`, na prática (destino `111122223333`):

- **ARN de role:** `arn:aws:iam::50000000000:role/eksWorkerNodeRole` vira `arn:aws:iam::111122223333:role/eksWorkerNodeRole`. O nodegroup `ng-01` passa a usar esse ARN traduzido como `node_role_arn`.
- **Policy customer-managed:** a role `ClusterAutoscalerRole` anexa `arn:aws:iam::50000000000:policy/ClusterAutoscalerPolicy`; como é uma policy sua (não da AWS), ela é recriada por nome e o anexo passa a usar `arn:aws:iam::111122223333:policy/ClusterAutoscalerPolicy`.
- **Policy gerenciada pela AWS:** `arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy` (e as demais `:aws:policy/...` da `eksWorkerNodeRole`) não mudam, porque o `aws` no lugar da conta é universal.
- **Token `__DEST_ACCOUNT__`:** a trust da `AmazonEKSFargatePodExecutionRole` tem a condição `aws:SourceArn = arn:aws:eks:xx-xxxx-1:50000000000:fargateprofile/*/*`; o `50000000000` ali é reescrito para `__DEST_ACCOUNT__` na captura e, no apply, para `111122223333`.
- **Saneamento:** as roles deste cluster têm trust limpa (`eks.amazonaws.com`, `ec2.amazonaws.com`, `eks-fargate-pods.amazonaws.com` e, nas roles de pod, `pods.eks.amazonaws.com`), então não há principal órfão nem provedor OIDC para remover aqui. Em um cluster que usasse IRSA, é nesta etapa que o provedor OIDC da origem seria retirado.

O que foi alterado por role é listado no `CLUSTER-INFO.md`.

9. O `vpc_found` como interruptor-mestre da rede

Este é o fato comportamental mais importante da solução, e o que mais afeta os cenários.

```
vpc_found = disable_resource_reuse ? false : (lookup VPC != "")
```

Toda a camada de rede é condicionada a `vpc_found`. Quando a VPC é encontrada (reutilizada), nada de rede é criado:

- subnets → procuradas (não criadas);
- route tables, rotas, associações, IGW, NAT, DHCP options, gateway/interface endpoints, attachment de TGW → todos com `for_each = vpc_found ? {} : (...)` → nenhum criado.

A premissa por trás disso: se a VPC com aquele CIDR já existe no destino, a malha de rede dela (route tables, IGW, NAT, attachments) também já existe, então o Terraform não recria roteamento, só reaproveita a VPC e garante subnets/SGs. Quando a VPC não é encontrada (`vpc_found=false`), a rede inteira é construída do zero a partir do backup.

Consequência prática direta nos cenários:

vpc_found	O que acontece com a rede
true (VPC reusada)	Nada de rede é tocado; só o cluster + filhos + IAM (reusado)
false (VPC nova)	Rede inteira recriada (subnets, RTs, rotas, IGW, NAT, SGs, endpoints, TGW)

Security groups e subnets seguem o `vpc_found` : VPC nova → criados; VPC reusada → procurados. IAM (roles/users/policies/profiles), key pairs e launch templates são independentes do `vpc_found` (dependem só de `disable_resource_reuse`), porque IAM é global e key pairs/LTs são regionais, não atrelados àquela VPC.

O cluster EKS (`aws_eks_cluster.main`) e seus filhos (nodegroups, fargate, add-ons, access entries, pod identities) são sempre criados, são o que você está recriando, não recursos de "base" sujeitos a find-or-create.

No `cluster-xxxx` : a VPC é `vpc-xxxx` (`x.x.x.x/xx`). Se ela for reutilizada no destino (`vpc_found = true`), as dezenas de subnets, as 4 route tables (`rt-public`, `-private-1a`, `-private-1b`, `-private-1c`), os 3 NAT gateways, o IGW `igw-xxxx`, o gateway endpoint do S3, o interface endpoint e o attachment do TGW ficam intocados; o apply só recria o cluster e seus filhos. Se ela for nova (`vpc_found = false`), tudo isso é reconstruído do backup (com EIPs novos nos 3 NATs).

10. Transit Gateway cross-account

O TGW tem um find-or-create próprio (em `transit_gateway.tf`), ativo apenas quando `recreate_external_routes = true` e a VPC é nova (`_tgw_enabled = recreate_external_routes && ! vpc_found`). Para cada TGW referenciado, o lookup resolve o ID no destino em cascata:

- 1. Mesmo ID via AWS RAM:** `describe-transit-gateways --transit-gateway-ids <id-origem>` encontra (TGW compartilhado, mesmo ID visível) → usa direto.
- 2. Mesmo Name tag:** procura um TGW com o mesmo Name, owned pela conta de destino, no estado `available` → usa o ID existente.
- 3. Não encontrado** → o Terraform cria um TGW novo com a mesma configuração da origem (ASN, `dns_support`, `vpn_ecmp_support`, `multicast_support`, route tables customizadas e rotas estáticas blackhole).

O ID efetivo (encontrado ou criado) vai para `tgw_id_by_source` e é usado pelo attachment da VPC e pelas rotas → `tgw-xxx`. O TGW criado recebe a tag-marca (lookup o ignora → idempotente) e uma tag `eks-importer:source-id` com o ID de origem.

No `cluster-xxxx` : o TGW é `tgw-xxxx` (Name `xxxx-TGW`, ASN `xxxxx`). Numa VPC nova, a etapa 1 tenta `describe-transit-gateways --transit-gateway-ids tgw-xxxx` (acha se o TGW estiver compartilhado via RAM com o destino, mesmo ID); a etapa 2 procura um TGW Name= `xxxx-TGW` próprio da conta de destino; e, se nada bater, a etapa 3 cria um TGW novo com `amazon_side_asn = xxxxx` e os mesmos options. O attachment entra com as subnets do attachment de origem (`subnet-xxxx`, `subnet-xxxx`, `subnet-xxxx`), e as muitas rotas → `tgw-xxxx` das 4 route tables (ex.: `x.x.x.x/xx`, `x.x.x.x/xx`, `x.x.x.x/xx`) passam a apontar para o ID resolvido. Como o seu TGW tem `route_tables: {}` (sem route tables customizadas), nada além do attachment e das rotas precisa ser recriado.

O que é capturado/recriado: options do TGW, Name + tags, route tables customizadas (não a default-association) e suas rotas estáticas blackhole.

Limitações do TGW (documentadas no `CLUSTER-INFO.md`):

- quando o TGW é criado novo, o attachment usa a default route table (associação e propagação ligadas por padrão), o que cobre o caso de uma única VPC; associações/propagações customizadas não são recriadas;

- outros attachments do TGW de origem (outras VPCs, VPN, Direct Connect, peering de TGW) não são recriados, só o attachment desta VPC;
- rotas estáticas que apontam para um attachment não são recriadas (só as blackhole); rotas propagadas (BGP) reaparecem sozinhas após os attachments;
- se o TGW for encontrado (RAM/Name), ele é reutilizado como está, a tool não altera a config nem as route tables dele.

11. O que é capturado e recriado (cobertura)

Domínio	Recursos	Find-or-create?
Rede, VPC	VPC, CIDRs secundários, DHCP options customizado	sim (VPC por CIDR)
Rede, subnets	todas as subnets (modo VPC inteira) ou só as do cluster	sim (por cidr)
Rede, roteamento	route tables, rotas (igw/nat/eoigw/tgw/peering/vgw/eni/carrier/core), associações	criados se VPC nova
Rede, gateways	Internet Gateway, NAT (zonal 1-por-subnet ou regional multi-AZ), egress-only IGW	criados se VPC nova
Rede, endpoints	Gateway (S3/DynamoDB) e Interface (ECR/STS/Logs/EC2...)	criados se VPC nova
Rede, Transit Gateway	config + route tables customizadas + rotas blackhole + attachment	sim (cascata, \$10)
IAM	roles (trust/managed/inline), policies customer-managed, users (de access entries)	sim (por nome)
Compute	Launch Templates, instance profiles, key pairs, Node Groups, Fargate, Auto Mode	LT/profile/keypair: sim
Acesso	Access Entries + policy associations, modo de autenticação	cluster-children
Add-ons	EKS Add-ons + Pod Identity Associations (de add-on e standalone)	cluster-children
Segurança	Security Groups adicionais + regras; regras manuais do Cluster SG	SGs: sim

O que isso significa no `cluster-xxxx` (o que o backup deste cluster contém): 3 node groups (`ng-01/02/03`), 4 fargate profiles (`fp-01` , `fp-02` , `fp-03` , `fp-04`), 6 add-ons (vpc-cni, coredns, kube-proxy, eks-pod-identity-agent, EBS e EFS CSI), 3 access entries (`app-user` , `brian.richard` , `gitlab-cluster-xxxx`), 6 pod identity associations (3 standalone + 3 de add-on), 9 IAM roles, 3 policies customer-managed (`ClusterAutoscalerPolicy` e as duas do Load Balancer Controller), 3 IAM users, 1 launch template (`lt-workers-xxxx`), 1 key pair (`keypair-xxxx`) e 3 security groups adicionais (`sg-app-1` , `sg-app-2` , `sg-workers`). Tudo isso é recriado; o que é find-or-create (VPC, subnets, SGs, roles, policies, key pair, LT, TGW) é reaproveitado quando já existe no destino.

O Cluster Security Group é criado pelo próprio EKS (não é especificável no Terraform); apenas as regras manuais adicionadas a ele são recriadas no Cluster SG do cluster novo. O user-data dos launch templates é removido de propósito (o EKS injeta o bootstrap do cluster novo); o original fica salvo em `launch-templates-userdata/`.

12. Cenários de uso

Esta é a parte central. Para cada cenário: o contexto, como configurar as flags, e o passo a passo recurso-a-recurso do que acontece no `terraform apply`.

Antes, fixe os dois fatos-chave que governam tudo:

1. `vpc_found` decide a rede inteira (§9): VPC reusada → rede intocada; VPC nova → rede reconstruída.
2. A tag-marca garante idempotência (§6): reaplicar não destrói o que a stack criou.

12.1 Cenário A: mesma conta, só o cluster foi deletado

Contexto: o cluster EKS foi deletado (por engano ou de propósito), mas a VPC, as subnets, o roteamento, o IGW/NAT, os SGs adicionais, as IAM roles e o TGW continuam existindo na mesma conta. É o "apaguei meu cluster, preciso de volta".

Flags: `disable_resource_reuse = false` (padrão), `recreate_external_routes = true` (padrão).

Não use `disable_resource_reuse = true` aqui: como as IAM roles e a VPC sobreviveram, forçar criação daria `EntityAlreadyExists` /conflito. O padrão `false` reusa o que sobreviveu.

O que acontece no apply, recurso a recurso:

Recurso	Lookup no destino	Resultado
VPC	acha a VPC (mesmo CIDR, sem marca)	<code>vpc_found = true</code> → reutiliza
Subnets, route tables, rotas, IGW, NAT, DHCP, endpoints	n/a (gateados por <code>!vpc_found</code>)	nada recriado (a malha já existe)
Attachment de TGW	n/a (gateado por <code>vpc_found</code>)	não recriado (a VPC já está anexada)
Security groups adicionais	procurados por (vpc-id + nome)	reutilizados (se sobreviveram)
IAM roles (cluster/node/fargate/pod-identity)	achadas por nome (sem marca)	reutilizadas (ARN idêntico, mesma conta)
IAM policies customer-managed	achadas por nome	reutilizadas
Key pairs / launch templates	achados por nome	reutilizados
Transit Gateway	estratégia 1 acha (mesma conta, mesmo ID)	<code>tgw_found = true</code> → reutilizado
Cluster EKS	n/a (sempre criado)	recriado
Cluster SG (gerenciado pelo EKS)	n/a	criado pelo EKS; regras manuais recriadas
Node Groups / Fargate / Add-ons / Access Entries / Pod Identities	n/a (filhos do cluster)	recriados

Resultado líquido: a rede e o IAM são 100% reaproveitados; apenas o cluster e seus filhos são recriados, exatamente como você quer. Tempo de apply curto, sem recriar VPC/NAT (sem custo nem downtime de rede).

No `cluster-xxxx`, na prática: a VPC `x.x.x.x/xx`, as 4 route tables, os 3 NATs, o TGW `xxxx-TGW` e as 9 roles (`eksServiceRole`, `eksWorkerNodeRole`, etc.) sobrevivem e são reaproveitados; o apply recria só o cluster

`cluster-xxxx`, os 3 node groups, os 4 fargate profiles, os 6 add-ons, as 3 access entries e as 6 pod identity associations. Como as roles de pod já usam `pods.eks.amazonaws.com`, o IRSA/OIDC não é problema aqui.

Pontos de atenção:

- O Cluster SG novo tem um ID diferente do antigo. Se algum SG externo referenciava o Cluster SG antigo pelo ID literal, ajuste para o novo (output `cluster_security_group_id`).
- Pod Identity/IRSA: a trust foi saneada; se alguma role dependia do provedor OIDC do cluster antigo (IRSA), o provedor do cluster novo é diferente, migre para Pod Identity (a trust já está pronta) ou recrie o IRSA provider.

12.2 Cenário B: mesma conta, tudo foi deletado

Contexto: "nuke", o cluster e a VPC, subnets, roteamento, IGW/NAT, SGs e (possivelmente) IAM roles foram deletados na mesma conta. O TGW e backbone persistente (VGW, core network) provavelmente sobreviveram (são infra compartilhada); se o TGW também foi apagado, ver abaixo.

Flags: `disable_resource_reuse = false` (padrão), recomendado, pois lida com "alguns sobreviveram, outros não". `recreate_external_routes = true`.

Você pode usar `disable_resource_reuse = true` se o destino estiver realmente limpo (nada da base sobrou). Mas se uma IAM role sobreviveu (IAM é global e some só se você apagar explicitamente), `true` daria `EntityAlreadyExists`. Na dúvida, `false`.

O que acontece no apply:

Recurso	Lookup	Resultado
VPC	não acha (foi deletada)	<code>vpc_found = false</code> → cria (com marca)
Subnets	n/a (VPC nova → criadas)	criadas todas, remapeadas por AZ
Route tables, rotas, associações	n/a	criadas (igw/nat/eoigw apontam para os recursos novos)
Internet Gateway	n/a	criado (se a origem tinha)
NAT Gateways	n/a	criados FIEL (zonal 1-por-AZ com EIP novo, ou regional multi-AZ)
DHCP options customizado	n/a	criado e associado (se a origem usava)
Gateway/Interface endpoints	n/a	criados (ECR/STS/Logs vitais p/ VPC privada)
Security groups adicionais	n/a (VPC nova → criados)	criados com as regras
IAM roles	achadas (se sobreviveram) / não achadas	reutilizadas ou criadas
Transit Gateway (sobreviveu)	estratégia 1 acha (mesmo ID)	reutilizado; attachment da VPC nova é criado
Transit Gateway (também foi apagado)	estratégia 1 falha, 2 (Name) falha	estratégia 3 cria um TGW novo idêntico
Rotas para TGW	n/a	criadas, apontando para o TGW resolvido
Rotas para peering/VGW/carrier	lookup no destino	find-or-skip: se o alvo existir na conta, a rota é criada; senão é pulada (sem falhar)

Recurso	Lookup	Resultado
Rotas para ENI	n/a	sempre puladas (ENI de appliance/LB; ELB recria as suas)
Rotas para core network	n/a	ARN literal: funciona se o Cloud WAN estiver compartilhado via RAM
Cluster + filhos	n/a	recriados

Resultado líquido: a infraestrutura inteira é reconstruída do backup. Os EIPs dos NAT são novos (se algum IP de saída era whitelistado externamente, atualize a allowlist). Se o TGW foi recriado (estratégia 3), ele é um TGW isolado, você precisa reconfigurar os outros attachments dele (VPN/DX/outras VPCs) manualmente.

12.3 Cenário C: outra conta, landing zone com base existente

Contexto: DR/migração para uma conta diferente que já tem uma landing zone pré-provisionada (uma VPC com o mesmo CIDR, talvez IAM roles padronizadas com os mesmos nomes) que você quer reaproveitar.

Flags: `disable_resource_reuse = false` (padrão). `recreate_external_routes`: depende da conectividade externa (ver §7), frequentemente `false` em cross-account se a malha (TGW/peering) será montada à parte, ou `true` se o TGW é RAM-shared/mesmo-nome.

Pré-requisitos cross-account:

- Preencher `aws_profile` (ou usar credenciais de ambiente) da conta de destino.
- Se a origem cifrava secrets com KMS: informar `cluster_encryption_kms_key_arn` (uma chave da conta de destino) ou `cluster_encryption_enabled = false`, uma precondition cobra isso.
- AMIs custom privadas da origem não existem no destino → troque por AMIs válidas lá (ou compartilhe as AMIs entre contas).

O que acontece no apply:

Recurso	Lookup no destino	Resultado
VPC	acha a VPC da landing zone (mesmo CIDR, sem marca)	<code>vpc_found = true</code> → reutiliza
Subnets	procuradas por (vpc-id + cidr) na VPC da landing zone	reutilizadas se os CIDRs baterem
Rede (RTs, IGW, NAT, endpoints, attachment)	n/a (gateado por <code>vpc_found</code>)	nada recriado (assume-se que a landing zone já tem)
Security groups adicionais	procurados por (vpc-id + nome)	reutilizados se existirem; criados se não
IAM roles	achadas por nome (se a landing zone padroniza nomes)	reutilizadas; ARN reescrito p/ a conta de destino
IAM roles inexistentes no destino	não achadas	criadas com trust saneada + account-id reescrito
Políticas customer-managed	por nome	reutilizadas ou recriadas (account-id → destino)
Cluster + filhos	n/a	recriados na conta de destino, ARNs/ subnets/SGs traduzidos

Resultado líquido: o cluster é recriado dentro da VPC e das roles da landing zone do destino. Como `vpc_found = true`, a rede da landing zone é assumida pronta, isso exige que os CIDRs de subnet do backup

batam com os da landing zone (senão a busca de subnet por CIDR falha e o cluster ficaria sem subnets válidas). Se não batem, ou você ajusta os CIDRs no tfvars, ou usa uma VPC de CIDR diferente (forçando `vpc_found=false` e a criação completa).

12.4 Cenário D: outra conta, conta limpa

Contexto: DR/migração para uma conta nova e vazia (sem VPC, sem roles, sem nada do cluster).

Flags: `disable_resource_reuse = false` (padrão) funciona, como nada casa, tudo é criado. `true` também funciona (conta limpa, sem risco de conflito). Em cross-account para conta limpa, ambos chegam ao mesmo resultado. `recreate_external_routes` : ver §7 (decida pelo TGW/peering).

O que acontece no apply:

Recurso	Lookup	Resultado
VPC	não acha	cria (com marca), <code>vpc_found = false</code>
Rede inteira (subnets, RTs, rotas, IGW, NAT, DHCP, endpoints, SGs)	n/a	criada do zero
IAM roles/users/policies/profiles	não achadas	criadas; trust saneada; account-id e ARNs reescritos p/ destino
Key pairs / launch templates	não achados	criados (chave pública da origem; user-data removido)
Transit Gateway (<code>recreate_external_routes=true</code>)	RAM/Name não acham	estratégia 3 cria um TGW novo + attachment da VPC
Rotas peering/VGW/carrier (se houver)	lookup no destino	find-or-skip: criadas se o alvo existir na conta nova; senão puladas (não falham)
Rotas ENI	n/a	sempre puladas (recriadas pelo LB Controller, se forem de ELB)
Rotas core network	n/a	ARN literal; ok se o Cloud WAN estiver compartilhado via RAM com a conta
Cluster + filhos	n/a	recriados na conta nova

Resultado líquido: ambiente inteiro nasce na conta nova a partir do backup. Pontos de atenção do cross-account todos se aplicam: KMS (chave do destino), AMLs custom, EIPs novos nos NAT, TGW isolado (estratégia 3) sem os outros attachments, peering cross-account manual, e provedor OIDC/IRSA inexistente (migre para Pod Identity).

12.5 Tabela-resumo dos cenários

	A: mesma conta, só cluster	B: mesma conta, tudo	C: outra conta, landing zone	D: outra conta, limpa
<code>disable_resource_reuse</code>	false	false	false	false (ou true)
<code>recreate_external_routes</code>	true	true	true / false (§7)	true / false (§7)
VPC	reusada	criada	reusada (landing zone)	criada

	A: mesma conta, só cluster	B: mesma conta, tudo	C: outra conta, landing zone	D: outra conta, limpa
Rede (RTs/NAT/IGW/endpoints)	intocada	recriada	intocada	recriada
IAM roles	reusadas (ARN igual)	reusadas/criadas	reusadas/criadas (ARN reescrito)	criadas (ARN reescrito)
TGW	reusado (mesmo ID)	reusado ou novo	RAM/Name ou novo	novo (ou §7)
Cluster + filhos	recriados	recriados	recriados	recriados
KMS dest. obrigatória?	não (mesma chave)	não	sim, se cifrado	sim, se cifrado
EIPs de NAT	n/a (não recria)	novos	n/a	novos

13. Passo a passo operacional do apply

```
cd terraform-<cluster>/ # (CLI) ou o diretório do commit no GitLab (Lambda)
```

```
# 1) Ajuste o tfvars para o DESTINO:
# - "aws_profile": profile da conta de destino (ou "" p/ cadeia padrão)
# - se cifrado por KMS: "cluster_encryption_kms_key_arn" do destino
# - revise "recreate_external_routes" conforme §7
# - (outra região) ajuste "availability_zone" das subnets
```

```
terraform init
terraform plan # LEIA o plano: o que é data (reusado) vs resource (criado)
terraform apply
```

Como ler o plan: recursos que o find-or-create encontrou aparecem resolvidos via `data.external.*` (reusados, sem bloco de criação). Os que faltam aparecem como `resource ... will be created`. Confira especialmente: a VPC vai ser criada ou reusada? (decide a rede inteira). As IAM roles estão sendo criadas ou reusadas? O TGW vai ser encontrado ou criado?

Ordem de dependências (garantida por `depends_on` no HCL): IAM roles/policies e associações de subnet → cluster; cluster → nodegroups → add-ons → pod identities; TGW attachment → rotas de TGW.

14. Estrutura dos arquivos gerados

```
<saída>/
├─ main.tf           # provider AWS, versões, default_tags
├─ variables.tf      # todas as variáveis (inclui transit_gateways)
├─ locals.tf         # tabelas de tradução, stack_marker_tags
├─ network.tf        # VPC, subnets, rotas, NAT, IGW, endpoints, attachment
├─ transit_gateway.tf # find-or-create do TGW (cascata RAM/Name/cria)
├─ iam.tf            # roles, users, policies, caller_identity, traduções
├─ security_groups.tf # SGs adicionais, default SG, regras, Cluster SG rules
├─ launch_templates.tf # LTs, instance profiles, key pairs (find-or-create)
├─ cluster.tf        # aws_eks_cluster (+ Auto Mode, KMS, preconditions)
├─ nodegroups.tf     # managed node groups (regras de LT da API do EKS)
├─ fargate.tf        # fargate profiles
├─ access.tf         # access entries + policy associations
├─ addons.tf         # add-ons + pod identity association
├─ pod_identity.tf   # pod identity associations standalone
├─ outputs.tf        # cluster_name/endpoint/arn/cluster_sg_id/...
├─ terraform.auto.tfvars.json # TODA a configuração capturada da origem
├─ scripts/
└─ aws_lookup.py     # helper do data.external (find-or-create no destino)
```

```
└─ launch-templates-userdata/ # user-data original dos LTs (referência; removido do LT)
└─ CLUSTER-INFO.md           # relatório: o que foi capturado + avisos + limitações
```

No tfvars, as duas flags de controle ficam no topo, juntas: `disable_resource_reuse` seguido de `recreate_external_routes`.

15. Permissões IAM

Captura (conta de ORIGEM, somente leitura): ações `Describe*` / `Get*` / `List*` em EKS, EC2, IAM, incluindo as três de Transit Gateway:

- `ec2:DescribeTransitGateways`
- `ec2:DescribeTransitGatewayRouteTables`
- `ec2:SearchTransitGatewayRoutes`
- `ec2:DescribeTransitGatewayVpcAttachments`

(No Lambda, adicione `secretsmanager:GetSecretValue` e `kms:Decrypt` se o token do GitLab vier de um secret cifrado.)

Aplicação (conta de DESTINO): as mesmas permissões de leitura (porque o `aws_lookup.py` roda no `plan`) mais as de criar/alterar/excluir todos os recursos que podem ser criados: EKS, EC2 (VPC/subnets/RTs/NAT/IGW/SG/endpoints/TGW), IAM (roles/policies/users/instance-profiles), e KMS se houver criptografia. O find-or-skip de rotas externas precisa ainda destas leituras no destino: `ec2:DescribeVpnGateways`, `ec2:DescribeVpcPeeringConnections` e `ec2:DescribeCarrierGateways`.

Em outras palavras, no destino a role precisa de permissão de escrita ampla; na origem, apenas leitura.

16. Idempotência e reexecução

Rodar `terraform apply` de novo (mesmo state, mesmo destino) é seguro: a tag-marca faz os lookups ignorarem o que a stack criou, então esses recursos continuam gerenciados pelo state, sem recriação nem destruição (§6). Recursos que você adicionou manualmente fora da stack (e que casem com as chaves de busca) continuam sendo reaproveitados.

Mudanças na origem entre capturas (ex.: novo nodegroup) aparecem no tfvars da próxima geração e o apply as concilia normalmente.

`disable_resource_reuse = true` também é idempotente (o Terraform sempre gerencia tudo), mas só em destino limpo, ver §7.

17. Limitações conhecidas

O `CLUSTER-INFO.md` gerado lista as limitações específicas do seu cluster. As principais, de forma consolidada:

- **Cluster Security Group:** gerenciado pelo EKS, não especificável no Terraform; apenas as regras manuais são recriadas (as default do EKS, não, ele as recria).
- **Transit Gateway criado novo:** usa a default route table (cobrindo VPC única); associações/propagações customizadas e outros attachments (VPN/DX/outras VPCs) não são recriados; só rotas estáticas blackhole são recriadas.

- **VPC Peering cross-account:** a conexão em si é manual (exige aceitação do outro lado). Mas a rota é resolvida por find-or-skip: se já existir uma peering ativa no destino que cubra o CIDR, a rota aponta para ela; senão é pulada (não falha o apply).
- **Rotas para peering / VGW / carrier:** find-or-skip. São consultadas na conta de destino; se o equivalente existir, a rota é criada apontando para ele; se não, a rota é pulada (o apply não falha). Funciona cross-account. Crie/aceite o peering ou anexe o VGW/carrier no destino para materializar a rota.
- **Rotas para ENI:** sempre puladas. A ENI pertence a um appliance/LB que a stack não gerencia. ENIs de ELB (requester `amazon-elb`) são recriadas pelo Load Balancer Controller e não precisam de rota estática; se for appliance seu, recrie-o e adicione a rota manualmente.
- **Rotas para core network (Cloud WAN):** usam o ARN literal. Funciona quando o core network está compartilhado via RAM com a conta de destino (caso comum); se não estiver, o apply falha nessa rota (compartilhe ou remova a rota).
- **NAT capturado (regional e zonal), com 3 exceções de borda:** o tool captura e recria fiel tanto o NAT regional (multi-AZ) quanto o NAT zonal público (1 por subnet, com EIP novo). Não são recriadas apenas as rotas que apontam para um NAT que não pôde ser capturado: (1) NAT já deletado na origem (a rota virou blackhole), (2) NAT zonal privado (`ConnectivityType=private`, sem EIP, pulado de propósito), e (3) NAT zonal cuja subnet não foi capturada (só ocorre no modo "subnets do cluster apenas"; o modo "VPC inteira" o captura). Nesses casos a rota é pulada e a subnet pode ficar sem saída (sem ECR); o relatório lista cada rota afetada (`orphan_nat_routes`).
- **KMS:** a chave da origem não existe no destino → informe uma do destino ou desligue a criptografia.
- **AMIs custom privadas:** não existem em outra conta → troque por AMIs válidas.
- **User-data dos LTs:** removido (o EKS injeta o bootstrap novo); o original fica em `launch-templates-userdata/`.
- **OIDC/IRSA:** o provedor do cluster antigo/origem não existe no destino; migre as roles para Pod Identity (a trust já é preparada) ou recrie o IRSA provider.
- **EIPs de NAT:** novos a cada recriação de VPC; atualize allowlists externas.
- **Mesma região:** AZ names preservados; outra região exige ajustar `availability_zone` no tfvars.
- **Landing zone (cenário C):** a reutilização da VPC exige que os CIDRs de subnet do backup batam com os da VPC de destino.

18. Troubleshooting / FAQ

O plan mostra que vai DESTRUIR a VPC/roles que eu quero manter. Provavelmente `disable_resource_reuse = true` num destino que não está limpo, ou o state está apontando para recursos que mudaram. Em recuperação na mesma conta com base sobrevivente, use `false` (find-or-create reusa). Releia §7 e §9.

apply falha com `EntityAlreadyExists` / `InvalidGroup.Duplicate`. `disable_resource_reuse = true` tentando criar algo que já existe. Mude para `false`.

Uma rota externa não apareceu no destino (peering/VGW/carrier). Não é erro: peering, VGW e carrier são find-or-skip. Se o equivalente não existe na conta de destino, a rota é pulada (não falha o `apply`). Crie/aceite o peering ou anexe o VGW/carrier no destino e rode o `apply` de novo, ou adicione a rota manualmente. O `CLUSTER-INFO.md` lista quais rotas externas estavam no backup.

apply falha numa rota de core_network (Cloud WAN). O core network usa ARN literal. Compartilhe o core network com a conta de destino via AWS RAM (aí o mesmo ARN passa a valer) ou remova essa rota do tfvars.

plan / apply trava ou falha nos `data.external`. O `aws_lookup.py` precisa do AWS CLI instalado e autenticado na conta de destino na máquina que roda o Terraform. Verifique credenciais e o `aws_profile` no tfvars.

O cluster ficou sem puxar imagens do ECR (nós NotReady). VPC privada sem rota de saída/endpoints adequados. Verifique os interface endpoints (ECR/STS/Logs) e as rotas de NAT; veja a seção de NAT (exceções de borda) no relatório.

A precondition reclama de `cluster_encryption_kms_key_arn`. A origem cifrava secrets com KMS. Informe uma chave KMS da conta de destino ou defina `cluster_encryption_enabled = false`.

Para que serve a pasta `__pycache__` (no pacote Python)? Cache de bytecode do Python; seguro deletar, recriado no import, sem efeito no Terraform. Está no `.gitignore`.